ĐẶC TẢ THIẾT KẾ gemm_main_fsm

Module của `gemm_accel_top` · instance `u_main_fsm`

Mục lục

1 Tổng quan **2**
1.1 Tính năng chính . . . 2
1.2 Tham số . . . 2
1.3 Tín hiệu giao tiếp . . . 2

2 Đặc tả thanh ghi **7**

3 Mô tả chức năng **7**
3.1 Tổ chức chức năng . . . 7
3.2 Dựng và kiểm tra job . . . 8
3.3 Bank và cấp quyền bắt đầu pha . . . 9
3.4 Engine . . . 11
3.5 Điều khiển tác vụ . . . 12
3.6 Timing . . . 14
3.7 Reset, hiện thực, kiểm chứng . . . 15

1 Tổng quan

1.1 Tính năng chính

Main FSM là module duy nhất ra lệnh cho DMA và array. Mỗi phép GEMM là một **job** lấy từ một descriptor. Ba engine độc lập đưa job qua các pha: **Đọc** (FETCH, kiểm tra, LOAD_*), **Tính** (COMPUTE) và **Ghi** (STORE_C). Bộ chọn bank quyết định khi nào một pha được bắt đầu, nên ba pha của ba job liên tiếp chạy chồng nhau mà không đụng cùng một bank. Module nằm trong miền `clk` duy nhất (100 MHz, chu kỳ 10,0 ns).

Chức năng

- Mỗi descriptor là một job: FETCH → kiểm tra → LOAD_BIAS (nếu `POST_EN`) → LOAD_A → LOAD_B (trừ khi `REUSE_B`) → COMPUTE → STORE_C. Chuỗi dừng ở descriptor có `LAST` = 1. Chuỗi descriptor là chế độ duy nhất
- Mọi tham số của job (kích thước, địa chỉ, stride, cờ, lượng tử) lấy từ descriptor 16 word do DMA đọc về. Register File chỉ cấp địa chỉ descriptor đầu chuỗi `i_cfg_desc_ptr`
- Ba engine chạy đồng thời khi điều kiện bank và thứ tự job cho phép
- Nội dung DRAM sau một tác vụ DONE giống hệt thực thi tuần tự từng job, với mọi thời gian trả lời của bus
- FSM báo kết thúc tác vụ cho Register File bằng đúng một xung done hoặc một xung error; điều phối soft reset; cấp bộ đếm `DESC_DONE_CNT`, thanh ghi `ERR_DESC_ADDR` và tín hiệu đếm hiệu năng
- FSM cấp cấu hình của job đang tính cho array và VPU, và bank đang tính cho scratchpad. `DESC_PTR` chỉ đi tới FSM

> **QUYẾT ĐỊNH BASELINE D-01**
> Không có chế độ lập trình bằng thanh ghi: FSM không có cổng `i_start_kind` và không có `i_cfg_dim`, `i_cfg_a_addr`, `i_cfg_b_addr`, `i_cfg_c_addr`, `i_cfg_post_ctrl`, `i_cfg_bias_addr`, `i_cfg_qm0`, `i_cfg_qshift`. Giao diện Register File → FSM chỉ còn `i_start_pulse`, `i_soft_req`, `i_cfg_desc_ptr`. Mọi `i_start_pulse` khởi động chuỗi: `R_START` → `R_FETCH` luôn luôn.

1.2 Tham số

Bảng 1.1: Tham số

Tham số	Mặc định	Ý nghĩa
`P`	8	Kích thước array; giá trị hợp lệ duy nhất là 8 (D-04). Chỉ dùng cho `HW_CONFIG` và assertion
`NB`	2	localparam cố định: hai bank mỗi buffer (D-09). Không có đường xử lý nào cho NB khác 2

1.3 Tín hiệu giao tiếp

Bảng 1.2: Tín hiệu giao tiếp của `gemm_main_fsm`

Tên tín hiệu	Độ rộng	Hướng	Mô tả
Clock, reset, soft reset			
`clk`	1	Input	Clock của khối. Các thanh ghi cập nhật tại cạnh lên.

Bảng 1.2: Tín hiệu giao tiếp của `gemm_main_fsm` (tiếp theo)

Tên tín hiệu	Độ rộng	Hướng	Mô tả
`rst_n`	1	Input	Reset tích cực mức thấp; được kích hoạt bất đồng bộ. Cạnh nhả reset được đồng bộ với clock tại top trước khi đưa tới các module (nối vào `rst_n_sync`).
`o_soft_clr`	1	Output	Xung xóa đồng bộ một chu kỳ gửi đến Register File, DMA, SRAM, array và VPU. Chỉ phát sau khi cả hai kênh DMA đã rảnh; hủy trạng thái tác vụ hiện tại. FSM không có cổng `i_soft_clr`: tự xóa bằng trạng thái T_CLR.
`o_soft_busy`	1	Output	Báo đang xử lý soft reset. Trong khoảng này Register File báo BUSY = 1 và không nhận tác vụ mới.
Register File → FSM			
`i_start_pulse`	1	Input	Xung một chu kỳ báo Register File vừa chấp nhận một tác vụ. `DESC_PTR` đã chụp hợp lệ cùng chu kỳ.
`i_soft_req`	1	Input	Xung một chu kỳ yêu cầu Main FSM thực hiện soft reset. Phát khi lệnh ghi CTRL có SOFT_RESET = 1 và byte 0 được cho phép ghi.
`i_cfg_desc_ptr`	32	Input	Bản chụp đủ 32 bit của DESC_PTR. Địa chỉ descriptor đầu chuỗi. Chụp khi chấp nhận tác vụ và giữ nguyên đến lần chấp nhận tiếp theo.
FSM → Register File			
`o_done_pulse`	1	Output	Xung một chu kỳ báo toàn bộ tác vụ kết thúc thành công. Register File đặt DONE và xóa BUSY tại cạnh lên; không xuất hiện cùng err pulse.
`o_err_pulse`	1	Output	Xung một chu kỳ báo toàn bộ tác vụ kết thúc với lỗi. Mã lỗi hợp lệ cùng chu kỳ; Register File đặt ERROR và xóa BUSY tại cạnh lên.
`o_err_code`	4	Output	Mã lỗi của tác vụ, được lấy mẫu cùng err pulse. Giá trị 1: cấu hình. Giá trị 2: đọc dữ liệu. Giá trị 3: ghi kết quả; 4: đọc descriptor. Giá trị 5 … 15 dự trữ, không bao giờ xuất hiện.
`o_perf_load`	1	Output	Báo pha LOAD dữ liệu, không bao gồm FETCH đang hoạt động. Khi tín hiệu bằng 1 và tác vụ đang bận, bộ đếm hiệu năng tương ứng tăng tại cạnh lên.
`o_perf_comp`	1	Output	Báo pha COMPUTE đang hoạt động. Khi tín hiệu bằng 1 và tác vụ đang bận, bộ đếm hiệu năng tương ứng tăng tại cạnh lên.
`o_perf_store`	1	Output	Báo pha STORE_C đang hoạt động. Khi tín hiệu bằng 1 và tác vụ đang bận, bộ đếm hiệu năng tương ứng tăng tại cạnh lên.
`o_err_desc_addr`	32	Output	Địa chỉ descriptor lỗi được cung cấp cho thanh ghi ERR_DESC_ADDR tại offset 0x44.

Bảng 1.2: Tín hiệu giao tiếp của `gemm_main_fsm` (tiếp theo)

Tên tín hiệu	Độ rộng	Hướng	Mô tả
`o_desc_done_cnt`	32	Output	Số descriptor đã hoàn tất STORE_C thành công, cung cấp cho thanh ghi DESC_DONE_CNT tại offset 0x48.
Lệnh kênh đọc DMA			
`o_rd_start`	1	Output	Xung một chu kỳ yêu cầu bắt đầu lệnh đọc DRAM. Chỉ phát khi kênh rảnh. Các tham số của lệnh được chốt tại cạnh lên của chu kỳ có o_rd_start = 1.
`o_rd_dst`	2	Output	Chọn đích của lệnh đọc. Giá trị 0: buffer A. Giá trị 1: buffer B. Giá trị 2: buffer BIAS. Giá trị 3: descriptor trả về Main FSM. Giá trị được chốt cùng xung o_rd_start.
`o_rd_base`	32	Output	Địa chỉ byte của hàng đầu tiên trong DRAM cho lệnh đọc DRAM. Địa chỉ phải căn 4 byte; được chốt cùng xung o_rd_start.
`o_rd_rows`	6	Output	Số hàng cần truyền trong lệnh đọc DRAM, từ 1 đến 32. Giá trị được chốt cùng xung o_rd_start.
`o_rd_rwords`	6	Output	Số word 32 bit cần truyền trong mỗi hàng của lệnh đọc DRAM, từ 1 đến 32. Giá trị được chốt cùng xung o_rd_start.
`o_rd_stride`	16	Output	Khoảng cách tính bằng byte giữa địa chỉ đầu hai hàng liên tiếp trong DRAM. Phải là bội của 4 và không nhỏ hơn 4 lần số word mỗi hàng. Được chốt cùng xung o_rd_start.
`o_rd_bank`	1	Output	Chọn bank đích của buffer A, B hoặc BIAS khi nạp dữ liệu; không dùng cho FETCH. Giá trị được chốt cùng xung o_rd_start.
`o_rd_abort`	1	Output	Yêu cầu dừng lệnh đọc DRAM; đây là tín hiệu mức. DMA ghi nhớ yêu cầu trong suốt lệnh, không khởi tạo burst mới và hoàn tất các giao dịch đã cam kết. Abort không tự tạo lỗi.
`i_rd_busy`	1	Input	Báo kênh đọc DRAM đang bận. Bằng 1 từ chu kỳ sau khi nhận o_rd_start đến hết chu kỳ báo done hoặc error; không nhận lệnh mới trong khoảng này.
`i_rd_done`	1	Input	Xung một chu kỳ báo lệnh đọc DRAM kết thúc mà không có lỗi AXI. Không xuất hiện đồng thời với error. Lệnh bị abort vẫn báo done nếu không gặp lỗi.
`i_rd_error`	1	Input	Xung một chu kỳ báo lệnh đọc DRAM kết thúc có lỗi. Lỗi gồm phản hồi R khác OKAY hoặc RLAST sai vị trí. Không xuất hiện đồng thời với done; chỉ báo sau khi giao dịch đang dở đã được xử lý.

Bảng 1.2: Tín hiệu giao tiếp của `gemm_main_fsm` (tiếp theo)

Tên tín hiệu	Độ rộng	Hướng	Mô tả
`i_desc_valid`	1	Input	Báo có một word descriptor hợp lệ trong chu kỳ này. Chỉ dùng với lệnh FETCH; bên nhận lấy dữ liệu khi valid = 1, không có ready.
`i_desc_idx`	4	Input	Chỉ số word trong descriptor, từ 0 đến 15. Hợp lệ cùng chu kỳ với i_desc_valid.
`i_desc_data`	32	Input	Nội dung 32 bit của word descriptor có chỉ số i_desc_idx. Hợp lệ cùng chu kỳ với i_desc_valid; dữ liệu của FETCH bị lỗi không được dùng để thực thi job.
Lệnh kênh ghi DMA			
`o_wr_start`	1	Output	Xung một chu kỳ yêu cầu bắt đầu lệnh ghi DRAM. Chỉ phát khi kênh rảnh. Các tham số của lệnh được chốt tại cạnh lên của chu kỳ có o_wr_start = 1.
`o_wr_fmt`	1	Output	Chọn định dạng dữ liệu trong buffer C. Giá trị 0: INT32, mỗi phần tử chiếm một word. Giá trị 1: INT8, bốn phần tử trong một word. Giá trị được chốt cùng xung o_wr_start.
`o_wr_base`	32	Output	Địa chỉ byte của hàng đầu tiên trong DRAM cho lệnh ghi DRAM. Địa chỉ phải căn 4 byte; được chốt cùng xung o_wr_start.
`o_wr_rows`	6	Output	Số hàng cần truyền trong lệnh ghi DRAM, từ 1 đến 32. Giá trị được chốt cùng xung o_wr_start.
`o_wr_rwords`	6	Output	Số word 32 bit cần truyền trong mỗi hàng của lệnh ghi DRAM, từ 1 đến 32. Giá trị được chốt cùng xung o_wr_start.
`o_wr_stride`	16	Output	Khoảng cách tính bằng byte giữa địa chỉ đầu hai hàng liên tiếp trong DRAM. Phải là bội của 4 và không nhỏ hơn 4 lần số word mỗi hàng. Được chốt cùng xung o_wr_start.
`o_wr_bank`	1	Output	Chọn bank nguồn của buffer C khi thực hiện STORE_C. Giá trị được chốt cùng xung o_wr_start.
`o_wr_abort`	1	Output	Yêu cầu dừng lệnh ghi DRAM; đây là tín hiệu mức. DMA ghi nhớ yêu cầu trong suốt lệnh, không khởi tạo burst mới và hoàn tất các giao dịch đã cam kết. Abort không tự tạo lỗi.
`i_wr_busy`	1	Input	Báo kênh ghi DRAM đang bận. Bằng 1 từ chu kỳ sau khi nhận o_wr_start đến hết chu kỳ báo done hoặc error; không nhận lệnh mới trong khoảng này.
`i_wr_done`	1	Input	Xung một chu kỳ báo lệnh ghi DRAM kết thúc mà không có lỗi AXI. Không xuất hiện đồng thời với error. Lệnh bị abort vẫn báo done nếu không gặp lỗi.

Bảng 1.2: Tín hiệu giao tiếp của `gemm_main_fsm` (tiếp theo)

Tên tín hiệu	Độ rộng	Hướng	Mô tả
`i_wr_error`	1	Input	Xung một chu kỳ báo lệnh ghi DRAM kết thúc có lỗi. Lỗi xảy ra khi phản hồi B khác OKAY. Không xuất hiện đồng thời với done; chỉ báo sau khi giao dịch đang dở đã được xử lý.
Array, VPU và scratchpad phía tính			
`o_pe_start`	1	Output	Xung một chu kỳ bắt đầu COMPUTE cho toàn bộ job. Chỉ phát khi array rảnh; kích thước và cấu hình tính toán phải hợp lệ cùng chu kỳ.
`i_pe_done`	1	Input	Xung một chu kỳ báo COMPUTE đã hoàn tất, bao gồm hậu xử lý và ghi kết quả vào buffer C. Không báo cho lượt tính bị soft clear hủy.
`i_pe_busy`	1	Input	Báo array đang tính. Bằng 1 từ chu kỳ sau khi nhận pe start đến hết chu kỳ có pe done; về 0 khi soft clear hủy lượt tính.
`o_comp_dim`	32	Output	Kích thước job: {8'h00, K[7:0], N[7:0], M[7:0]}. Bit [7:0]: M; [15:8]: N; [23:16]: K. M từ 1 đến 32; N và K là bội của 4, từ 4 đến 32. Phải hợp lệ khi bắt đầu lượt tính. Bit [31:24] luôn bằng 0; giữ nguyên từ pe start đến hết pe done.
`o_comp_post_en`	1	Output	Cho phép hậu xử lý: 0 xuất INT32 nguyên gốc; 1 cộng bias, lượng tử hóa và xuất INT8. Giữ ổn định từ pe start đến hết pe done.
`o_comp_relu_en`	1	Output	Cho phép chặn kết quả âm về 0 trước khi kẹp INT8. Bị bỏ qua khi hậu xử lý tắt. Giữ ổn định từ pe start đến hết pe done.
`o_comp_qm0`	32	Output	Hệ số nhân lượng tử M0, số nguyên 32 bit có dấu. Giữ ổn định từ pe start đến hết pe done.
`o_comp_qshift`	5	Output	Lượng dịch bổ sung sh, số không dấu từ 0 đến 31. Tổng lượng dịch trong công thức lượng tử là 31 + sh. Giữ ổn định từ pe start đến hết pe done.
`o_comp_bank_a`	1	Output	Chọn bank của buffer A dành cho phía tính. Giữ nguyên trong suốt COMPUTE.
`o_comp_bank_b`	1	Output	Chọn bank của buffer B dành cho phía tính. Giữ nguyên trong suốt COMPUTE.
`o_comp_bank_bias`	1	Output	Chọn bank của buffer BIAS dành cho phía tính. Giữ nguyên trong suốt COMPUTE.
`o_comp_bank_c`	1	Output	Chọn bank của buffer C dành cho phía tính. Giữ nguyên trong suốt COMPUTE.
`o_comp_active`	1	Output	Báo COMPUTE đang hoạt động. SRAM dùng tín hiệu này để kiểm tra quyền truy cập bằng assertion (AS-02 của SRAM); tín hiệu không thay thế các enable đọc/ghi.

Giao tiếp

- Mọi port `o_*` lấy từ thanh ghi, kể cả `o_rd_abort` và `o_wr_abort`; hai port này là tín hiệu mức (không phải xung), chỉ giải mã từ thanh ghi trạng thái của FSM: `o_rd_abort` = T_SOFT ∧ (engine Đọc ở R_FWAIT ∨ R_LWAIT), `o_wr_abort` = T_SOFT ∧ engine Ghi ở S_RUN. Không dùng `i_rd_busy`, `i_wr_busy`, nên không

có đường tổ hợp từ input tới hai port này

- Lệnh DMA: FSM chỉ bật `o_rd_start` khi kênh đọc rảnh (`i_rd_busy` = 0 và không có lệnh nào đang chờ `i_rd_done`/`i_rd_error`); tương tự kênh ghi. Các port tham số chỉ cần hợp lệ ở chu kỳ `o_*_start` = 1; DMA tự chốt
- `o_comp_dim`, `o_comp_post_en`, `o_comp_relu_en`, `o_comp_qm0`, `o_comp_qshift`, `o_comp_bank_*` hợp lệ từ chu kỳ `o_pe_start` = 1 tới hết chu kỳ `i_pe_done` = 1 và không đổi trong khoảng đó
- `o_pe_start` chỉ khi `i_pe_busy` = 0. Mỗi `o_pe_start` có đúng một `i_pe_done`
- `o_done_pulse` và `o_err_pulse` không bao giờ cùng chu kỳ; mỗi `i_start_pulse` có tối đa một trong hai; không có cái nào cho tác vụ bị soft reset hủy

QUYẾT ĐỊNH BASELINE D-10
`o_comp_dim` rộng 32 bit như spec_1, nội dung {8'h00, K, N, M}. Main FSM đã kiểm `DIM[31:24]` = 0 trước khi phát nên bit [31:24] luôn bằng 0; array bỏ qua [31:24].

2 Đặc tả thanh ghi

Module không có thanh ghi ánh xạ MMIO. Bản đồ thanh ghi (`CTRL`, `STATUS`, `HW_CONFIG`, `PERF_*`, `DESC_PTR`, `ERR_DESC_ADDR`, `DESC_DONE_CNT`) do `gemm_register_file` sở hữu và được định nghĩa trong mục Đặc tả thanh ghi của đặc tả hệ thống. Cấu hình của từng job được nhận từ descriptor qua `i_desc_*`; trạng thái nội bộ, điều kiện chốt và reset được mô tả bên dưới.

3 Mô tả chức năng

3.1 Tổ chức chức năng

Bảng 3.1: Module con

Module	Instance	Vai trò
`gemm_fsm_ctrl`	`u_ctrl`	Trạng thái tác vụ; khởi động chuỗi; bản ghi lỗi; tín hiệu chặn; cấp quyền bắt đầu pha; done/error; soft reset; `ERR_DESC_ADDR`
`gemm_job_check`	`u_chk` (trong engine Đọc)	Tổ hợp: dựng job, kiểm kích thước, địa chỉ, cờ, bit dự trữ và stride theo các điều kiện trong mục Dựng và kiểm tra job; tính stride hiệu dụng.
`gemm_rd_engine`	`u_rd`	Engine Đọc: FETCH, CHECK, chờ WAIT_PREV, LOAD_*, đẩy job vào slot
`gemm_cp_engine`	`u_cp`	Engine Tính: lấy job từ slot, chạy COMPUTE, giữ job tới khi engine Ghi nhận
`gemm_wr_engine`	`u_wr`	Engine Ghi: STORE_C; `DESC_DONE_CNT`; báo job cuối
`gemm_bank_sb`	`u_sb`	Chọn bank theo con trỏ của từng buffer; bộ đếm chiếm dụng bank (scoreboard)

Luồng job: engine Đọc → slot (một job) → engine Tính → (C_HAND) → engine Ghi. Có bốn máy trạng thái, mã hóa one-hot: điều khiển tác vụ (`gemm_fsm_ctrl`, 4 bit), engine Đọc (13 bit), engine Tính (3 bit), engine Ghi (2 bit). Trạng thái ban đầu sau reset là bit IDLE tương ứng.

QUYẾT ĐỊNH BASELINE D-29 (PHẦN TRỌNG TÀI ENGINE)
Khi nhiều engine cùng muốn bắt đầu pha ở một chu kỳ, thứ tự ưu tiên cố định là Ghi > Tính > Đọc. Ba engine dùng các tài nguyên khác nhau (kênh ghi DMA, array, kênh đọc DMA) nên trọng tài chỉ quyết định thứ tự cấp `go` khi `want` trùng chu kỳ.

3.1.1 Job

Một job là struct `gemm_pkg::job_t`. Engine Đọc giữ một job, slot giữ một job, engine Tính và engine Ghi mỗi bên giữ một job; tối đa bốn job cùng tồn tại.

Bảng 3.2: Trường của `job_t`

Trường	Rộng	Nội dung
`idx`	32	Số thứ tự trong chuỗi, 0, 1, 2, …
`daddr`	32	Địa chỉ descriptor (cho `ERR_DESC_ADDR`)
`m, n, k`	6 × 3	M, N, K (1 … 32)
`a, b, c, bias`	32 × 4	Địa chỉ byte của A, B, C/Y, bias
`lda, ldb, ldc`	16 × 3	Stride hiệu dụng (byte), đã thay 0 bằng độ dài hàng
`qm0, qsh`	32, 5	M_0, sh
`post, relu, reuse, waitp, last`	1 × 5	Cờ
`next`	32	`NEXT_PTR` (chỉ engine Đọc dùng)
`ba, bb, bbias, bc`	1 × 4	Bank của A, B, BIAS, C (gán ở CHECK)

3.2 Dựng và kiểm tra job

3.2.1 Từ descriptor

Descriptor dài 16 word = 64 byte, căn 64 byte. Engine Đọc nhận 16 word qua `i_desc_*` và lưu word 0 … 10 vào bộ đệm. Word 11 (`TAG`) được bỏ qua, mọi giá trị đều hợp lệ; word 12 … 15 chỉ cần OR dồn để kiểm bằng 0.

Bảng 3.3: Job từ descriptor

Trường	Giá trị (wn = word thứ n)
`m, n, k`	w0[7:0], w0[15:8], w0[23:16]; w0[31:24] = 0
`a, b, c`	w1, w2, w3
`lda, ldb`	w4[15:0], w4[31:16]; 0 → K, N
`ldc`	w5[15:0]; 0 → N nếu `post`, ngược lại 4N; w5[31:16] dự trữ
`last, waitp, reuse, post, relu`	w6[0], w6[1], w6[2], w6[3], w6[4]; w6[31:5] dự trữ
`next, bias, qm0, qsh`	w7, w8, w9, w10[4:0]; w10[31:5] dự trữ
`TAG`	w11, phần cứng không đọc
`idx, daddr`	Bộ đếm descriptor của engine Đọc; con trỏ dùng để FETCH

Kiểm tra descriptor (`gemm_job_check`)

- Hợp lệ khi và chỉ khi mọi điều sau đúng: $1 \le M \le 32$; N, K ∈ {4, 8, …, 32}; w0[31:24] = 0; w5[31:16] = 0; w6[31:5] = 0; w10[31:5] = 0; w12 … w15 = 0
- `a`[1:0] = 0 và `c`[1:0] = 0; `b`[1:0] = 0 nếu `reuse` = 0; `bias`[1:0] = 0 nếu `post` = 1
- Mỗi stride thô (w4[15:0], w4[31:16], w5[15:0]) bằng 0, hoặc là bội của 4 và không nhỏ hơn độ dài hàng: K cho A; N cho B; 4N cho C khi `post` = 0, N khi `post` = 1
- `relu` = 1 kéo theo `post` = 1
- `reuse` = 1 chỉ khi `idx` > 0 và (K, N) bằng (K, N) của job trước trong chuỗi (thanh ghi `prev_kn` của engine Đọc)
- `last` = 0 kéo theo `next`[5:0] = 0. Vi phạm được tính là lỗi của chính descriptor này

- DESC_PTR[5:0] ≠ 0 được kiểm ở gemm_fsm_ctrl khi bắt đầu chuỗi: lỗi mã 1 với f = 0, ERR_DESC_ADDR = DESC_PTR, không truy cập bus và không truy cập scratchpad

> **QUYẾT ĐỊNH BASELINE D-24 VÀ D-32**
> Phần cứng chỉ kiểm các điều kiện trên (D-32, gộp cả hai nguồn). Phần cứng không kiểm vùng nhớ có nằm trong DRAM hay không và không kiểm chồng lấn giữa vùng ghi C/Y với vùng A, B, bias; vi phạm cho kết quả không xác định, lỗi bus báo bằng mã 2, 3 hoặc 4. N, K bội của 4 do phần mềm đệm 0 (D-26); M không chia hết cho 8 do phần cứng xử lý trong tile biên.

Mã lỗi

Bảng 3.4: Mã lỗi ERR_CODE

Mã	Nguồn	Điều kiện
0	Không lỗi	Không có lỗi
1	CHECK, gemm_fsm_ctrl	Descriptor không hợp lệ, hoặc DESC_PTR[5:0] ≠ 0
2	LOAD_A / LOAD_B / LOAD_BIAS	i_rd_error khi nạp dữ liệu
3	STORE_C	i_wr_error
4	FETCH	i_rd_error khi đọc descriptor
5 … 15	Dự trữ	Không bao giờ xuất hiện

> **QUYẾT ĐỊNH BASELINE D-02**
> Mã lỗi 0 … 4. Mã 5 … 15 dự trữ. Spec_2 định nghĩa thêm mã 5 (“tính năng không có”); baseline bỏ vì chỉ có một chế độ (D-01).

3.3 Bank và cấp quyền bắt đầu pha

3.3.1 Chọn bank

gemm_bank_sb: chọn bank

- Thanh ghi con trỏ nxt_a, nxt_c, nxt_bias, nxt_b, prev_b: về 0 khi chấp nhận tác vụ, khi rst_n_sync và khi soft_clr
- Ở chu kỳ CHECK của job hợp lệ: ba=nxt_a, bc=nxt_c; bbias=nxt_bias nếu post (ngược lại 0); bb=prev_b nếu reuse, ngược lại bb=nxt_b và prev_b<=nxt_b
- Cùng cạnh clock đó đảo nxt_a, nxt_c; đảo nxt_bias nếu post; đảo nxt_b nếu reuse = 0

> **QUYẾT ĐỊNH BASELINE D-29 (PHẦN CHỌN BANK)**
> A và C đảo bank sau mỗi descriptor; B đảo sau mỗi lần có LOAD_B (REUSE_B = 1 dùng lại bank của descriptor trước); BIAS đảo sau mỗi descriptor có POST_EN = 1. Mỗi buffer có đúng hai bank (D-09).

3.3.2 Bộ đếm chiếm dụng

Bảng 3.5: Bộ đếm của gemm_bank_sb

Bộ đếm	Rộng	Tăng khi	Giảm khi
rcnt[A][b]	2	Engine Đọc bắt đầu LOAD_A vào bank b	COMPUTE của job có ba = b xong (i_pe_done)
rcnt[B][b]	2	LOAD_B vào bank b, hoặc job reuse = 1 có bb = b đi qua bước B	COMPUTE của job có bb = b xong
rcnt[BIAS][b]	2	LOAD_BIAS vào bank b	COMPUTE của job post = 1 có bbias = b xong

Bảng 3.5: Bộ đếm của `gemm_bank_sb` (tiếp theo)

Bộ đếm	Rộng	Tăng khi	Giảm khi
`wcnt[C][b]`	1	COMPUTE của job có `bc` = b bắt đầu	STORE_C của job đó xong OKAY (`i_wr_done`)

Điều kiện bank

- LOAD_X vào bank b chỉ được bắt đầu khi `rcnt[X][b]` = 0: mọi job đọc bank đó đã tính xong. Job `reuse` = 1 không chờ (dữ liệu B của bank đó vẫn còn, vì chưa có LOAD_B nào vào bank ấy kể từ job trước)
- COMPUTE vào bank C b chỉ được bắt đầu khi `wcnt[C][b]` = 0: STORE_C cuối cùng đọc bank đó đã xong
- Tăng và giảm cùng chu kỳ trên cùng một bộ đếm được cộng ròng: $+1 - 1$ = giữ nguyên. Trường hợp duy nhất có thể xảy ra là bước R_RB tăng `rcnt[B][bb]` đúng chu kỳ `i_pe_done` giảm cùng bộ đếm đó (job đang tính dùng cùng bank B). LOAD_X chỉ tăng khi `rcnt[X][b]` = 0 và COMPUTE chỉ tăng `wcnt[C][b]` khi bộ đếm = 0, nên hai trường hợp này không trùng với lần giảm trên cùng bộ đếm
- Bộ đếm không bao giờ tràn hay âm (assertion). `rcnt` tối đa 3 vì tối đa ba job đã gán bank mà chưa tính xong (engine Đọc, slot, engine Tính)
- Khi tác vụ kết thúc do lỗi, bộ đếm có thể khác 0; chúng được xóa khi chấp nhận tác vụ kế tiếp hoặc khi `soft_clr`
- Các điều kiện trên là điều kiện cần để bắt đầu pha; phần cứng được phép bắt đầu muộn hơn, không bao giờ sớm hơn

3.3.3 Cấp quyền bắt đầu pha

Mỗi engine tính tín hiệu `want` từ trạng thái đầu chu kỳ: `want_s` (Ghi), `want_c` (Tính), `want_r` (Đọc: FETCH hoặc một LOAD). `gemm_fsm_ctrl` trả `go`.

$$go_x = want_x \wedge \neg inh$$

`inh` = (ctrl không ở T_RUN) $\vee$ `i_soft_req`. Thành phần `i_soft_req` là bắt buộc: ở chu kỳ `i_soft_req` đến, ctrl vẫn còn ở T_RUN; nếu chỉ xét trạng thái thì `go` vẫn được cấp và `o_rd_start`, `o_wr_start` hoặc `o_pe_start` sẽ lên ở chu kỳ kế tiếp, khi ctrl đã ở T_SOFT (vi phạm AS-12). Ba engine được cấp quyền độc lập; thứ tự job do bộ đếm bank giữ. Điều kiện bắt đầu pha của job d:

- LOAD vào một bank bắt đầu khi COMPUTE cuối cùng đọc bank đó đã xong
- COMPUTE(d) bắt đầu khi mọi LOAD của d xong, COMPUTE(d$-$1) xong, và STORE_C cuối cùng đọc bank C của d xong
- STORE_C(d) bắt đầu khi COMPUTE(d) và STORE_C(d$-$1) xong
- `WAIT_PREV` = 1 chặn các pha LOAD của descriptor (chờ `desc_done_cnt` bằng `idx`); không chặn FETCH và kiểm tra

Chặn sau lỗi

- `gemm_fsm_ctrl` cấp `blk_v`, `blk_idx` tổ hợp: bản ghi lỗi hiện có, gộp với lỗi đến trong chính chu kỳ này (`i_rd_error`, `i_wr_error`, kết quả CHECK sai), lấy chỉ số nhỏ nhất
- Mọi `want` của job có `idx` $\geq$ `blk_idx` bằng 0 khi `blk_v` = 1. Nhờ gộp lỗi cùng chu kỳ, không pha nào của descriptor $\geq$ f bắt đầu, kể cả ở đúng chu kỳ lỗi được báo
- Pha đã bắt đầu thì chạy tới hết (không abort khi lỗi). Kết quả của chúng không được ghi ra DRAM vì STORE_C của job $\geq$ f không bao giờ bắt đầu

3.4 Engine

3.4.1 Lệnh DMA

Bảng 3.6: Tham số lệnh DMA của mỗi pha

Pha	dst / fmt	base	rows	rwords	stride	bank
FETCH	3 (desc)	con trỏ	1	16	64	0
LOAD_BIAS	2 (BIAS)	`bias`	1	N	4N	`bbias`
LOAD_A	0 (A)	`a`	M	K/4	`lda`	`ba`
LOAD_B	1 (B)	`b`	K	N/4	`ldb`	`bb`
STORE_C	`post`	`c`	M	`post` ? N/4 : N	`ldc`	`bc`

FETCH là một burst INCR 16 beat; descriptor không đi vào scratchpad mà DMA trả về qua `i_desc_valid/i_desc_idx/i_desc_data`.

3.4.2 Engine Đọc

Mã hóa one-hot 13 bit, thứ tự bit: R_IDLE, R_START, R_FETCH, R_FWAIT, R_CHECK, R_WAITP, R_LBIAS, R_LA, R_LB, R_RB, R_LWAIT, R_PUSH, R_STOP.

Bảng 3.7: Trạng thái của `gemm_rd_engine`

Trạng thái	Làm gì	Ra khỏi trạng thái khi	Sang
R_IDLE	Chờ	ctrl khởi động tác vụ (`i_start_pulse` ở T_IDLE)	R_START; R_STOP nếu `DESC_PTR`[5:0] $\neq 0$
R_START	Con trỏ $\leftarrow$ `DESC_PTR`, idx $\leftarrow 0$	Sau đúng 1 chu kỳ	R_FETCH (luôn luôn)
R_FETCH	`want_r` = 1; khi `go_r`: phát FETCH	`go_r` = 1	R_FWAIT
R_FWAIT	Nhận word descriptor; chờ `i_rd_done` / `i_rd_error`	`i_rd_done` = 1, hoặc `i_rd_error` = 1	R_CHECK; R_STOP (lỗi mã 4)
R_CHECK	1 chu kỳ: `gemm_job_check`; nếu hợp lệ, gán bank	Sau đúng 1 chu kỳ	R_WAITP; R_STOP (lỗi mã 1)
R_WAITP	Nếu `waitp` = 1: chờ `desc_done_cnt==idx`	`waitp` = 0, hoặc `desc_done_cnt` = `idx`	R_LBIAS nếu `post`, ngược lại R_LA
R_LBIAS	`want_r` = (`rcnt[BIAS][bbias]` = 0). Khi `go_r`: `rcnt[BIAS][bbias]++`, phát LOAD_BIAS	`go_r` = 1	R_LWAIT
R_LA	`want_r` = (`rcnt[A][ba]` = 0). Khi `go_r`: `rcnt[A][ba]++`, phát LOAD_A	`go_r` = 1	R_LWAIT
R_LB	`want_r` = (`rcnt[B][bb]` = 0). Khi `go_r`: `rcnt[B][bb]++`, phát LOAD_B	`go_r` = 1	R_LWAIT
R_RB	Job `reuse`: `rcnt[B][bb]++`, không phát lệnh	Sau đúng 1 chu kỳ	R_PUSH
R_LWAIT	Chờ `i_rd_done` / `i_rd_error`	`i_rd_done` = 1, hoặc `i_rd_error` = 1	Thứ tự BIAS $\rightarrow$ A $\rightarrow$ (B hoặc RB) $\rightarrow$ PUSH; R_STOP (lỗi mã 2)
R_PUSH	Khi slot trống: slot $\leftarrow$ job. Nếu `last` = 0: `prev_kn` $\leftarrow$ (K, N), con trỏ $\leftarrow$ `next`, idx++	Slot trống	R_FETCH nếu `last` = 0; R_STOP nếu `last` = 1
R_STOP	Không làm gì tới hết tác vụ	Reset, T_CLR, hoặc ctrl chuyển T_RUN $\rightarrow$ T_IDLE (DONE hoặc ERROR, D-34)	R_IDLE

Engine Đọc

- Ở mọi trạng thái chờ (R_FETCH, R_WAITP, R_LBIAS, R_LA, R_LB, R_PUSH): nếu job bị chặn thì sang R_STOP, không đẩy job vào slot
- Engine Đọc làm xong mọi LOAD của job d rồi mới FETCH job d+1. `WAIT_PREV` không chặn FETCH và CHECK
- Mỗi descriptor được FETCH đúng một lần, theo đúng thứ tự chuỗi (D-23)
- Bộ đệm descriptor ghi word `i_desc_idx` khi `i_desc_valid`. Dữ liệu của FETCH bị lỗi không được dùng
- `o_perf_load` = 1 khi ở R_LWAIT (không tính FETCH)
- Trình tự job thường: R_FETCH → R_FWAIT → R_CHECK → R_WAITP → (R_LBIAS) → R_LA → R_LB (hoặc R_RB) → R_PUSH → R_FETCH; mỗi LOAD đi qua R_LWAIT

3.4.3 Engine Tính và engine Ghi

Engine Tính mã hóa one-hot 3 bit (C_IDLE, C_RUN, C_HAND); engine Ghi one-hot 2 bit (S_IDLE, S_RUN).

Bảng 3.8: Trạng thái của `gemm_cp_engine` và `gemm_wr_engine`

Trạng thái	Làm gì	Ra khỏi trạng thái khi	Sang
C_IDLE	`want_c` = slot có job ∧ `wcnt[C][slot.bc]` = 0. Khi `go_c`: job ← slot, slot trống, `wcnt++`, `o_pe_start` = 1 ở chu kỳ kế tiếp	`go_c` = 1	C_RUN
C_RUN	`o_comp_active` = `o_perf_comp` = 1. Khi `i_pe_done`: giảm `rcnt[A][ba]`, `rcnt[B][bb]`, và `rcnt[BIAS][bbias]` nếu `post`	`i_pe_done` = 1	C_HAND
C_HAND	Giữ job (và bank C) cho engine Ghi	Engine Ghi nhận job (`go_s` = 1)	C_IDLE
S_IDLE	`want_s` = engine Tính ở C_HAND. Khi `go_s`: job ← job của engine Tính, phát STORE_C ở chu kỳ kế tiếp	`go_s` = 1	S_RUN
S_RUN	`o_perf_store` = 1. `i_wr_done`: `wcnt[C][bc]--`; `desc_done_cnt++`; nếu `last` = 1, báo job cuối xong. `i_wr_error`: lỗi mã 3	`i_wr_done` = 1, hoặc `i_wr_error` = 1	S_IDLE

VÌ SAO ENGINE TÍNH GIỮ JOB Ở C_HAND KHÔNG LÀM CHẬM
COMPUTE(d+1) dùng bank C của job d−1, nên theo điều kiện bắt đầu COMPUTE (mọi LOAD của d+1 xong, COMPUTE(d) xong, STORE_C cuối cùng đọc bank C của d+1 xong) chỉ bắt đầu được khi STORE_C(d−1) xong. Đúng lúc đó engine Ghi rảnh và nhận job d, nên engine Tính về C_IDLE và có thể bắt đầu d+1 ngay chu kỳ sau.

3.5 Điều khiển tác vụ

Mã hóa one-hot 4 bit, thứ tự bit: T_IDLE, T_RUN, T_SOFT, T_CLR; trạng thái reset là T_IDLE.

Bảng 3.9: Trạng thái của `gemm_fsm_ctrl`

Trạng thái	Làm gì	Ra khỏi trạng thái khi	Sang
T_IDLE	Chờ. `i_start_pulse`: xóa bản ghi lỗi, `desc_done_cnt`, `err_desc_addr`, con trỏ bank, bộ đếm; khởi động engine Đọc	`i_start_pulse` = 1; hoặc `i_soft_req` = 1	T_RUN; T_CLR
T_RUN	Engine chạy. Kiểm điều kiện kết thúc. Ở cạnh clock chuyển sang T_IDLE: dọn engine (D-34): engine Đọc → R_IDLE, engine Tính → C_IDLE, engine Ghi → S_IDLE, `slot_v` ← 0	`i_soft_req` = 1 (ưu tiên); hoặc điều kiện DONE hoặc ERROR đúng	T_SOFT; T_IDLE
T_SOFT	`inh` = 1; `o_rd_abort` = 1 khi engine Đọc ở R_FWAIT hoặc R_LWAIT, `o_wr_abort` = 1 khi engine Ghi ở S_RUN (tức kênh đó đang có lệnh); không báo done/err	Không engine nào chờ lệnh DMA	T_CLR
T_CLR	`o_soft_clr` = 1 trong 1 chu kỳ; mọi thanh ghi của FSM về giá trị reset	Sau đúng 1 chu kỳ	T_IDLE

Kết thúc tác vụ

- Bản ghi lỗi (`err_v`, `err_idx`, `err_code`, `err_addr`) chỉ được thay khi lỗi mới có idx nhỏ hơn. `err_addr` = `daddr` của job lỗi; = con trỏ đang FETCH với lỗi mã 4; = `DESC_PTR` khi `DESC_PTR`[5:0] $\neq$ 0 (lỗi mã 1 với f = 0)
- DONE: ở T_RUN, engine Ghi báo job có `last` = 1 xong OKAY và `err_v` = 0 → `o_done_pulse` ở chu kỳ kế tiếp. Chuỗi kết thúc ở descriptor đầu tiên có `LAST` = 1
- ERROR: ở T_RUN, `err_v` = 1 và hệ yên: engine Đọc ở R_STOP và không chờ DMA; engine Tính không ở C_RUN; slot trống hoặc có idx $\geq$ f; engine Tính không giữ ở C_HAND một job idx < f; engine Ghi ở S_IDLE → `o_err_pulse` với `o_err_code` = `err_code` ở chu kỳ kế tiếp; `o_err_desc_addr` ← `err_addr`
- Khi đó `o_desc_done_cnt` = f tự nhiên, vì mọi job < f đã ghi xong và không job $\geq$ f nào được ghi
- `DESC_DONE_CNT` tăng 1 cho mỗi STORE_C nhận phản hồi B cuối là OKAY
- Dọn engine khi kết thúc (D-34): ở cạnh clock ctrl rời T_RUN về T_IDLE (cùng cạnh phát `o_done_pulse` hoặc `o_err_pulse`), engine Đọc về R_IDLE, engine Tính về C_IDLE, engine Ghi về S_IDLE và `slot_v` ← 0. Job idx $\geq$ f còn trong slot hoặc ở C_HAND bị bỏ, không bao giờ được COMPUTE hay STORE_C. Việc dọn an toàn vì điều kiện DONE/ERROR đã bảo đảm không kênh DMA nào bận và engine Tính không ở C_RUN. Không có bước này, engine Đọc kẹt ở R_STOP và tác vụ kế tiếp không bao giờ FETCH (BUSY = 1 mãi), còn job cũ sót lại sẽ bị tính và ghi vào `C_ADDR` của tác vụ trước

> **QUYẾT ĐỊNH BASELINE D-23 VÀ D-11**
> `o_done_pulse` và `o_err_pulse` không bao giờ cùng một chu kỳ. Khi `DESC_PTR`[5:0] $\neq$ 0, ctrl ghi bản ghi lỗi (mã 1, f = 0) ngay chu kỳ `i_start_pulse`, engine Đọc đi thẳng R_IDLE → R_STOP, không có lệnh DMA nào được phát, và `o_err_pulse` xuất hiện ở chu kỳ t + 2 tính từ `i_start_pulse` ở chu kỳ t.

Soft reset

- `o_soft_busy` = 1 ở T_SOFT và T_CLR; trong cả chuỗi Register File đọc STATUS ra 0x0000_0002, START và SOFT_RESET ghi lặp bị bỏ qua
- Ở T_SOFT: không pha mới nào bắt đầu, không lệnh DMA mới; lệnh DMA đang chạy kết thúc bằng done hoặc error như thường (burst đã cam kết chạy hết); engine vẫn nhận các xung đó để rời trạng thái chờ, nhưng không cập nhật bản ghi lỗi hay báo done
- COMPUTE đang chạy không được chờ: `o_soft_clr` hủy array và VPU

- `i_soft_req` ở T_IDLE: T_CLR ở chu kỳ kế tiếp. `i_soft_req` ở T_SOFT hoặc T_CLR: bỏ qua
- `i_soft_req` ở T_RUN thắng điều kiện DONE/ERROR đúng trong cùng chu kỳ: ctrl sang T_SOFT, không phát `o_done_pulse` hay `o_err_pulse` (tác vụ kết thúc bằng soft reset, không DONE, không ERROR)
- Nếu `i_soft_req` đến cùng chu kỳ với `i_rd_done` hoặc `i_wr_done`, kênh đó coi như đã rảnh
- Sau `o_soft_clr`: STATUS = 0, không báo DONE, không báo ERROR; `DESC_PTR` và các thanh ghi RW giữ nguyên; nội dung scratchpad không xác định. Abort không phải lỗi

QUYẾT ĐỊNH BASELINE D-33
Trình tự: `CTRL`[1] = 1 → `o_soft_req` (thắng START trong cùng lệnh ghi) → T_SOFT (T_RUN) hoặc T_CLR (T_IDLE) → `o_soft_clr` đúng một chu kỳ tới Register File, DMA, SRAM, array, VPU → T_IDLE. FSM tự reset, không có `i_soft_clr`.

3.6 Timing

Bảng 3.10: Độ trễ cố định của FSM

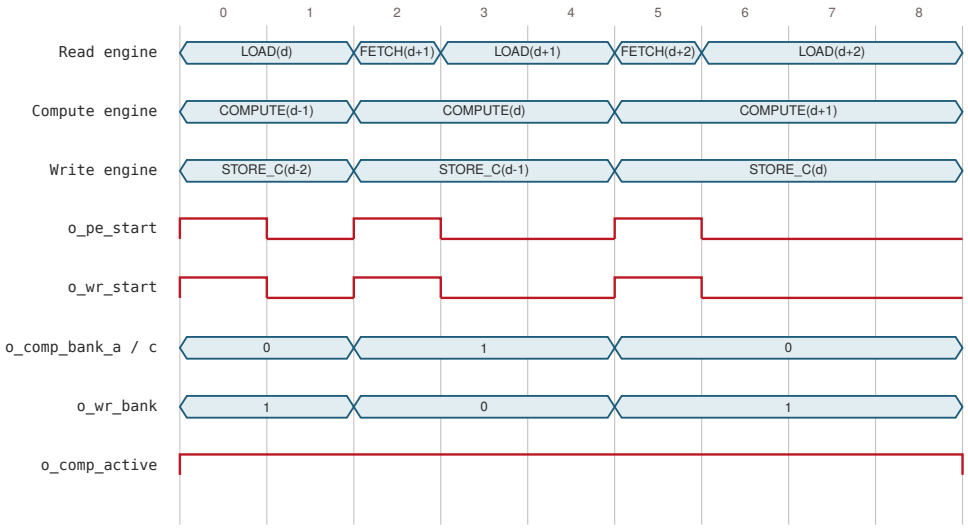
Sự kiện	Chu kỳ
`i_start_pulse` → engine Đọc ở R_START	+1 (10 ns)
R_CHECK	1 chu kỳ (10 ns)
`go_*` = 1 → `o_rd_start` / `o_wr_start` / `o_pe_start` = 1	+1 (10 ns)
`i_*_done` → trạng thái kế tiếp của engine	+1 (10 ns)
Điều kiện DONE / ERROR → `o_done_pulse` / `o_err_pulse`	+1 (10 ns)
`i_soft_req` ở T_IDLE → `o_soft_clr`	+1 (10 ns)
`i_start_pulse` → `o_err_pulse` khi `DESC_PTR`[5:0] $\neq 0$	+2 (20 ns)

Bảng dưới là số liệu của các module khác mà FSM phải tính đến khi lập lịch. Một chu kỳ ở 100 MHz là 10 ns.

Bảng 3.11: Thời gian các pha cho M = N = K = 32, `POST_EN` = 0, stride = 0

Pha	Thời gian
FETCH	16 beat
LOAD_A	32 × 8 = 256 beat
LOAD_B	32 × 8 = 256 beat
COMPUTE	534 chu kỳ = 5,34 µs
STORE_C	32 × 32 = 1024 beat
Đỉnh array (tham khảo)	64 MAC/chu kỳ × 100 MHz = 6,4 GMAC/s

QUYẾT ĐỊNH BASELINE D-06 VÀ D-25
COMPUTE_CYCLES = (NT − 1)·max(K, P) + K + P + r_last + LAT + 3, với r_last = (số hàng hợp lệ của tile cuối) − 1 và LAT = 4. Với M = N = K = 32: 15·32 + 32 + 8 + 7 + 4 + 3 = 534 chu kỳ. Số liệu thời gian nạp và ghi do người soạn baseline tính từ hai nguồn, ở mức 1 beat/chu kỳ và bỏ qua overhead burst. FETCH + nạp + ghi = 1552 beat ≈ 15,52 µs cho một descriptor chạy đơn lẻ, lớn hơn COMPUTE. Kênh đọc và kênh ghi AXI độc lập, nên khi chạy chồng với ping-pong thời gian mỗi descriptor tiến tới max(FETCH + nạp, tính, ghi) = max(528, 534, 1024) chu kỳ ≈ thời gian ghi: tác vụ 32^3 bị giới hạn bởi kênh ghi AXI 32 bit, không bởi array.



Hình 3.1: Chồng pha của chuỗi descriptor (trục thời gian không theo tỷ lệ; mỗi cột là một mốc, các pha thực có độ dài khác nhau). Ở mốc 2 … 5, FETCH(d+1) và LOAD(d+1) chạy cùng COMPUTE(d) trên bank `o_comp_bank_a` = `o_comp_bank_c` = 1 và STORE_C(d−1) đọc bank C = 0. LOAD(d+1) ghi vào bank A của d−1 (đã tính xong ở mốc 2). COMPUTE(d+1) bắt đầu ở mốc 5, đúng khi STORE_C(d−1) xong và bank C = 0 được giải phóng; STORE_C(d) bắt đầu cùng lúc khi COMPUTE(d) xong. Vì trục là mốc, các xung vẽ cùng cột không có nghĩa là cùng chu kỳ: ở mức chu kỳ, `o_wr_start` và `o_pe_start` của cùng một mốc lệch nhau ít nhất 1 chu kỳ (engine Tính chỉ rời C_HAND khi engine Ghi nhận job bằng `go_s`, rồi mới có `go_c` cho job kế tiếp), và `o_comp_active` có khoảng 0 ngắn giữa hai COMPUTE (C_HAND, C_IDLE); xem bảng trạng thái của engine Tính và engine Ghi

3.7 Reset, hiện thực, kiểm chứng

Reset

- `rst_n_sync` = 0 hoặc T_CLR: mọi engine về trạng thái IDLE, slot trống, bản ghi lỗi, con trỏ bank, bộ đếm, `desc_done_cnt`, `err_desc_addr`, mọi xung đầu ra về 0. Trường dữ liệu của job (`job_t`, bộ đệm descriptor) không reset và dùng `always_ff@(posedgeclk)`
- Flip-flop trạng thái dùng `always_ff@(posedgeclkornegedgerst_n_sync)`; T_CLR xóa đồng bộ qua dữ liệu, không qua chân reset

Hiện thực

- SystemVerilog tổng hợp được; FSM viết theo kiểu hai khối (`always_ff` trạng thái, `always_comb` trạng thái kế); không có latch. Cả bốn FSM mã hóa one-hot
- `job_t` khai báo trong `gemm_pkg` là `structpacked`; công cụ tổng hợp tự bỏ trường không dùng ở mỗi engine
- Mọi so sánh idx dùng 32 bit không dấu
- Không có bộ nhân trong FSM; đường tổ hợp dài nhất là `want` → `go` → trạng thái kế và `blk_v`/`blk_idx` (so sánh idx 32 bit), đủ cho chu kỳ 10 ns

Assertion bắt buộc (SVA)

Bảng 3.12: Danh sách assertion của `gemm_main_fsm`

Mã	Khẳng định
AS-01	`o_rd_start` → kênh đọc rảnh; `o_wr_start` → kênh ghi rảnh
AS-02	`o_pe_start` → `!i_pe_busy`; mỗi `o_pe_start` có đúng một `i_pe_done`
AS-03	Không bộ đếm `rcnt` hay `wcnt` nào tràn hay âm

Bảng 3.12: Danh sách assertion của `gemm_main_fsm` (tiếp theo)

Mã	Khẳng định
AS-04	LOAD_X bắt đầu vào bank b → không job nào ở slot, C_RUN hay C_HAND chưa tính xong có bank X = b (trừ job reuse; với X = BIAS chỉ xét job có `post` = 1, vì job `post` = 0 mang `bbias` = 0 nhưng không đọc BIAS, D-35)
AS-05	COMPUTE vào bank C b bắt đầu → `wcnt[C][b]` = 0
AS-06	STORE_C bắt đầu theo thứ tự idx tăng dần 1
AS-07	`o_done_pulse` ∧ `o_err_pulse` không bao giờ cùng lúc; mỗi `i_start_pulse` có tối đa một trong hai
AS-08	`o_comp_*` ổn định trong khi C_RUN; `o_comp_dim[31:24]` = 0
AS-09	Mỗi máy trạng thái (T_*, R_*, C_*, S_*) luôn là one-hot (`onehot(state)`)
AS-10	`o_err_pulse` → `o_err_code` ∈ {1, 2, 3, 4}
AS-11	`o_soft_clr` chỉ khi `i_rd_busy` = 0 và `i_wr_busy` = 0, và kéo dài đúng 1 chu kỳ
AS-12	T_SOFT hoặc T_CLR → không `o_rd_start`, `o_wr_start`, `o_pe_start`
AS-13	`blk_v` → không pha nào của job có idx ≥ `blk_idx` bắt đầu
AS-14	`o_done_pulse` → `err_v` = 0 và job `last` đã STORE_C xong OKAY
AS-15	T_IDLE → engine Đọc ở R_IDLE ∧ engine Tính ở C_IDLE ∧ engine Ghi ở S_IDLE ∧ ¬`slot_v` (D-34)

Kiểm tra chức năng

- Chuỗi 1 … 7 descriptor ngẫu nhiên (stride khác 0, `REUSE_B`, `WAIT_PREV`, `POST_EN`/`RELU_EN`): DRAM sau DONE khớp thực thi tuần tự từng descriptor
- Descriptor sai ở vị trí f: `ERR_CODE` = 1, `DESC_DONE_CNT` = f, `ERR_DESC_ADDR` = địa chỉ descriptor f, không pha nào của job ≥ f bắt đầu sau lỗi, các job < f chạy xong
- `DESC_PTR` lệch căn 64 byte: `ERR_CODE` = 1, không giao dịch AXI nào, `o_err_pulse` ở t + 2
- Lỗi AXI tiêm ở FETCH, LOAD_*, STORE_C cho mã 4, 2, 3 tương ứng
- Soft reset ở T_IDLE, ở T_RUN khi kênh đọc bận, khi kênh ghi bận, khi COMPUTE đang chạy: sau `o_soft_clr`, STATUS = 0, `DESC_PTR` giữ nguyên, không DONE, không ERROR
- Hai tác vụ liên tiếp: tác vụ 1 kết thúc ERROR khi slot hoặc C_HAND còn giữ job idx ≥ f, tác vụ 2 chạy chuỗi hợp lệ: tác vụ 2 DONE, không có ghi nào vào `C_ADDR` của tác vụ 1 sau lỗi; tương tự sau tác vụ DONE (D-34)
- Mô hình COMPUTE: `i_pe_done` cách `o_pe_start` đúng COMPUTE_CYCLES (534 chu kỳ với 32^3)

KIỂM CHỨNG BẰNG MÔ HÌNH

(Mô hình nằm ở `gemm-accel/model/`, đã chạy lại với baseline ngày 12/11/2026; kết quả đạt.) `gemm_sched_model.py` chạy 1 200 chuỗi ngẫu nhiên 1 – 7 descriptor (stride, `REUSE_B`, `WAIT_PREV` với descriptor đọc kết quả của descriptor trước, descriptor sai, lỗi AXI tiêm ngẫu nhiên ở FETCH/LOAD/STORE). Mọi bên đọc lấy dữ liệu ở đầu và cuối pha và phải thấy giống nhau, nên ghi đè sai bank bị phát hiện. Kết quả: DRAM khớp thực thi tuần tự ở mọi chuỗi DONE; với lỗi, `DESC_DONE_CNT` = f, `ERR_DESC_ADDR` đúng, không pha nào của job ≥ f bắt đầu sau lỗi. Mô hình đã bắt hai lỗi quy tắc trong quá trình thiết kế (đường `REUSE_B` và lỗi cùng chu kỳ), dẫn tới bước R_RB và tín hiệu `blk_v`, `blk_idx` tổ hợp.